

Shielded PPO: what we built and how it works

1. What we are doing

A single self-driving car has to cross a four-way junction with no traffic light, while cars keep arriving from the other three roads. It must decide, moment by moment, how hard to accelerate or brake so it crosses without hitting anyone and without waiting longer than necessary.

The car is driven by a neural network trained with reinforcement learning (PPO) in SUMO. The pre-defence report proposed adding a safety shield on top of it. Building, testing and reporting that shield is the work described here.

2. What the shield is

The shield is a set of fixed safety rules sitting between the network and the simulator. The network does not know about it: it simply proposes an acceleration. The shield checks that proposal against the nearby cars, then either passes it through unchanged or replaces it with a safer value, normally braking.

The aim is a safety guarantee that does not depend on learning.

3. How the shield works, step by step

Every simulation step, in this order:

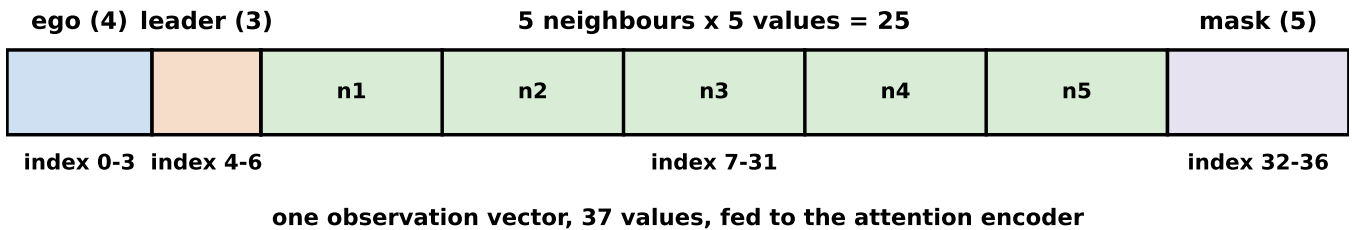
- The network outputs one number in $[-1, +1]$, turned into an acceleration: positive accelerates, negative brakes.
- Layer 1, time-to-conflict. For each nearby car the shield computes how long our car needs to reach the point where the two paths cross. If that is under 2 seconds and the other car arrives at about the same time, it brakes hard enough to stop short of that point.
- Layer 2, safe distance (RSS). It adds reaction distance, braking distance and a 3 metre margin. If our car is closer to the conflict point than that, it brakes.
- Layer 3, right of way. If two cars would arrive together and the other comes from the right, our car yields, because the junction uses right-before-left priority.
- Two later additions: the commit zone stops the shield braking once the car can no longer stop before the conflict point, and the rear-aware cap limits braking when a car is close behind.
- The surviving value is sent to SUMO. Every intervention is counted per layer, so we can report how often the shield acts and which rule fired.

4. Training and evaluation setup

Algorithm	PPO (stable-baselines3); attention encoder producing 256 values, then actor and critic of 256×256 .
Hyper-parameters	learning rate $3e-4$ to $1e-5$, batch 8192, minibatch 256, 10 epochs, discount 0.98, GAE 0.95, clip 0.25, entropy 0.01.
Training	1.5 million simulation steps per controller, 8 parallel simulators.
Evaluation	6 traffic levels x 4 turning patterns x 42 episodes = 1,008 episodes per controller, policy deterministic. Same protocol as the pre-defence report.

State space: what the car sees (37 numbers)

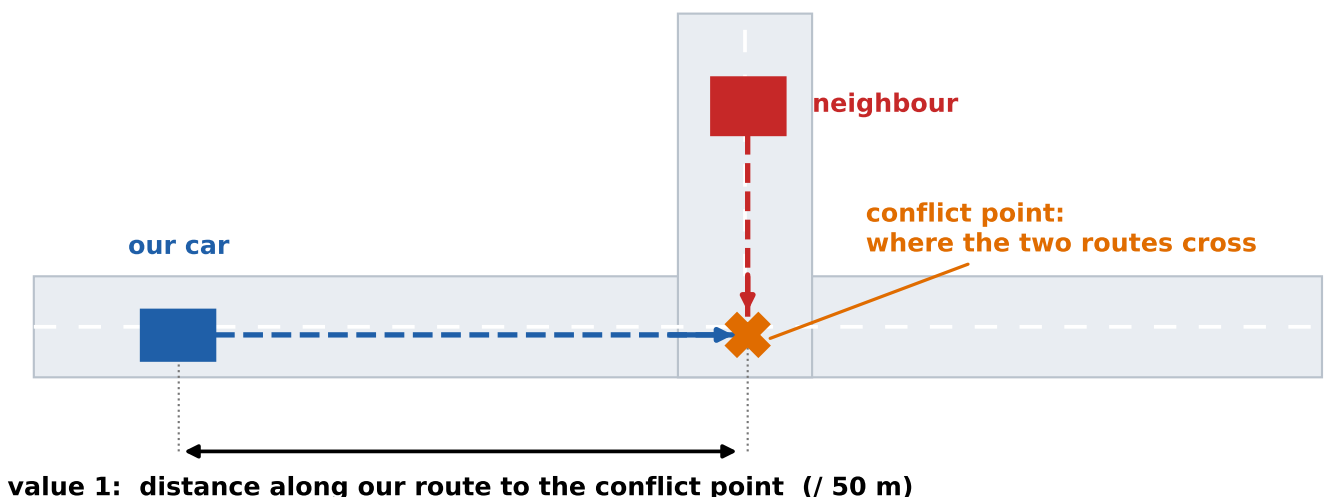
The network receives one flat vector of 37 numbers each step. Every value is normalised, so they all sit roughly between -1 and +1. The car looks 50 metres around itself and keeps the five closest vehicles.



ego (4 values)	distance still to drive to the exit as a fraction of the route, speed / 55, and our heading as sin and cos.
leader (3 values)	the car directly ahead in our own lane: gap / 50 m, its speed / 55, and time-to-collision with it. All three are 1.0 when the lane ahead is empty.
each neighbour (5)	distance to the point where our two paths cross (/ 50 m), its speed / 55, the difference in arrival time at that point, and its heading relative to ours as sin and cos.
mask (5 values)	1 if that neighbour slot holds a real car, 0 if it is empty padding. The attention layer ignores the 0 slots.

The two values that make this work

Each neighbour is not described by its raw position on the map, but by where our two routes cross and when each of us gets there:



value 2: arrival-time difference = (our time to reach the X) - (their time to reach the X)

close to 0 -> both cars arrive together, a collision course

large -> one car is safely through before the other arrives

Measuring the distance to the shared conflict point rather than the straight-line distance between the cars is what makes left turns work: two cars can be far apart and still head for the same spot at the same instant. These two values also feed the shield's rules and the reward's safety term.

Action space and reward function, as implemented

5. Action space

The action is one continuous number per step; the car does not steer, and its route is fixed when it enters the network.

$$a_t \in \mathcal{A} = [-1, +1] \subset \mathbb{R} \quad \text{one continuous value per step}$$

That number is mapped to a physical acceleration with two different scales, because the car can brake harder than it can accelerate:

$$u_t = a_{\max} \cdot a_t = 2.6 a_t \quad \text{if } a_t \geq 0 \text{ (accelerate)}$$

$$u_t = b_{\max} \cdot a_t = 4.5 a_t \quad \text{if } a_t < 0 \text{ (brake)}$$

So $a = +1$ is full acceleration at 2.6 m/s^2 , $a = -1$ is full emergency braking at 4.5 m/s^2 , and $a = 0$ holds speed. The command u is exactly what the shield inspects, and may replace, before the car executes it.

Discrete version for comparison we also trained a five-way action, a in $\{-1, -0.5, 0, +0.5, +1\}$, mapped with the same two scales. Coarser control, but an easier learning problem.

6. Reward function

The reward paid at each step is:

$$r_t = -10 \quad \text{if a collision occurs (episode ends)}$$

$$r_t = +15 \quad \text{if the car reaches its exit (episode ends)}$$

$$r_t = 10 \Delta p_t + s_t - 0.01 \quad \text{otherwise, at every step}$$

with the progress term and the safety term defined as:

$$\Delta p_t = p_t - p_{t-1} \quad \text{fraction of the route covered this step}$$

$$s_t = - \sum_{j \in N_t} e^{-10 |\Delta \tau_j|} - e^{-10 \text{TTC}_t} \quad \begin{array}{l} \text{first sum over neighbours with } |\Delta \tau_j| < 0.2, \\ \text{second term only if } \text{TTC}_t < 0.2 \end{array}$$

Meaning of the terms

Δp is progress towards the exit, $\Delta \tau$ for neighbour j is the gap between our arrival time and theirs at their shared conflict point, and TTC is the time-to-collision with the car directly ahead.

-10 and +15

the only large rewards: crashing must be worse than being slow, and reaching the exit is the goal.

Progress term

a full crossing sums to about +10, so the car is rewarded for approaching the exit rather than for raw speed.

Safety term

-1.0 per neighbour when the two arrival times are identical, about -0.14 at a difference of 0.2, and zero beyond it. This punishes near-misses long before any crash happens.

Time cost

the constant -0.01 per step stops the car from waiting at the junction indefinitely.

Shielded version

a further -0.005 whenever the shield has overridden more than half of the recent steps, to discourage leaning on it.

Results: where it worked and where it failed

7. Where it worked: our training setup

Here the surrounding traffic obeys the simulator's safety rules, so those cars brake for our car. Traffic arrives at 275 vehicles/hour on the crossing roads and the car's own lane is clear.

Shield off	0.99% of episodes ended in a collision (10 of 1,008).
Shield on	0.40% (4 of 1,008), travel time 12.9 to 16.3 seconds, 18.5% of actions overridden.
For comparison	heuristic controllers without attention 3.17% and 3.97%; best controller in the pre-defence study 0.88%.

Why it worked: the shield's only tool is braking, and braking is safe when the cars around you yield. Slowing down removed crossing conflicts without creating new ones.

Limitation: 10 collisions against 4 is not statistically significant ($p = 0.18$). All three attention controllers sit between 0.40% and 0.99% and cannot be separated at this sample size, so the shield is in the best group but not proven the best.

8. Where it failed: the pre-defence scenario

That setup is harsher: 400 vehicles/hour on the crossing roads, traffic in the car's own lane, and background drivers that ignore the simulator's safety checks, so they never brake. We retrained and tested the controller in that environment.

Shield off	6.35% collisions (64 of 1,008).
Shield on	9.72% (98). Significantly worse, $p = 0.007$.
Rear-aware cap	9.42% (95). Still significantly worse, $p = 0.013$.
+ commit zone	6.35% (64). Back to the unshielded level, $p = 1.00$, with overrides down from 8.8% to 2.9%.
Trained with shield	12.70%, but training was cut to 824k steps by the deadline against 1.5M for the baseline, so this is inconclusive.

Why it failed: the cars behind do not brake. When the shield slows our car for a crossing car, the car behind drives into it. The damage was worst on left turns, where the shield brakes most: 11.1% without it against 23.8% with it. Only the commit zone, which stops the shield braking inside the junction, removed the harm.



The shield can only brake, and braking is safe only if the car behind brakes too.

It does in our setup; in the pre-defence setup it does not, so the crash moves to the rear.

9. Conclusion and next step

A braking-only shield added on top of a trained controller helps when the surrounding traffic cooperates and hurts when it does not. The refined shield is safe in both environments but does not improve on the controller alone. The next step is to train with the shield on from the start, on a matched budget for a fair comparison.